

EduGenie

Google Gemini Powered Learning Assistant

Phase 7 – Project Documentation

Complete Project, Technical and User Documentation

Document Date: September 23, 2026

SmartBridge Phase 7 Documentation

Table of Contents

1. Project Overview	3
2. Problem Statement	4
3. Proposed Solution	4
4. Project Objectives	5
5. Project Scope	5
6. Technologies Used	6
7. System Architecture	7
8. System Workflow	8
9. Project Structure	9
10. Installation and Setup	10
11. Configuration	11
12. Features and Functionality	12
13. User Guide	14
14. AI / Gemini Integration	15
15. API Documentation	16
16. System Workflow Diagram	17
17. Testing Summary	18
18. Security and Privacy Considerations	18
19. Limitations	19
20. Future Enhancements	20
21. Conclusion	20

1. Project Overview

EduGenie is an AI-powered educational assistant built with FastAPI and Google Gemini. The application helps students and learners enhance their educational experience by providing intelligent support for questions, concept explanations, quiz generation, content summarization, and personalized learning path recommendations.

Purpose: EduGenie addresses the need for accessible, intelligent tutoring support by leveraging generative AI to provide contextual, student-friendly learning assistance.

Technology Stack: The application uses FastAPI as the backend framework, JavaScript/HTML/CSS for the frontend interface, and Google Gemini API for AI-powered content generation.

Target Users: Students at all educational levels, from beginner to advanced learners seeking personalized learning support.

2. Problem Statement

Traditional educational models often struggle to provide personalized, on-demand learning support. Students frequently encounter situations where they need:

- Instant answers to conceptual questions
- Simplified explanations of complex topics
- Practice quizzes for self-assessment
- Concise summaries of large educational materials
- Guidance on structured learning paths

Existing solutions are often expensive, require subscriptions, or lack the flexibility to adapt to individual learning styles. EduGenie addresses these challenges by providing a free, open-source, intelligent learning companion.

3. Proposed Solution

EduGenie leverages Google Gemini's generative AI capabilities to deliver intelligent, context-aware learning support. The solution is structured as a client-server application with clear separation of concerns:

Frontend

A modern, intuitive web interface built with HTML5, CSS3, and Vanilla JavaScript. Users select learning functions, input their content or questions, and receive AI-generated responses in real-time. The interface includes a task selector, input area, and live result display.

Backend Processing

A FastAPI-based backend that handles HTTP requests, validates input, routes requests to appropriate learning modules, and manages communication with the Gemini API. The backend ensures API keys remain secure on the server side.

Learning Modules

Five specialized learning modules process requests and generate tailored AI prompts: Question & Answer (QA), Concept Explanation, Quiz Generation, Text Summarization, and Learning Path Recommendations.

AI Integration

Google Gemini 2.5 Flash model processes prompts with configurable temperature settings to balance creativity and accuracy. Each learning module uses optimized prompt engineering to deliver high-quality, educational responses.

4. Project Objectives

- Develop an AI-powered learning assistant using Google Gemini
- Provide multiple learning functions (Q&A, explanations, quizzes, summaries, learning paths)
- Create an intuitive user interface for student interaction
- Implement secure API key management
- Demonstrate generative AI integration in educational technology
- Provide a scalable foundation for future educational features

5. Project Scope

Current Scope (Implemented)

- Question & Answer functionality with contextual responses
- Topic explanation with beginner-friendly content
- MCQ quiz generation (1–10 questions)
- Text summarization for educational passages
- Personalized learning path recommendations
- RESTful API with 5 core endpoints
- Web-based user interface with real-time response display
- Environment-based configuration for API keys

Target Users

Students, educators, learners, and educational technology professionals seeking an open-source learning assistant with AI capabilities.

Current Use Cases

- A student asking subject-specific questions and receiving clear, educational answers
- A learner requesting an explanation of a complex concept in simple terms
- An educator generating multiple-choice quizzes from educational materials
- A researcher summarizing lengthy academic passages quickly

- A learner requesting a structured learning path for a new topic

Boundaries (Not Implemented)

- User accounts and persistent learning history
- Real-time progress tracking
- Mobile-native applications (web-based only)
- Offline functionality
- Database persistence of user data
- Advanced analytics and reporting
- Video content generation
- Multi-language support (English only)

6. Technologies Used

Technology	Purpose
Python 3.10+	Backend development language
FastAPI 0.115+	High-performance web framework
Uvicorn	ASGI server for FastAPI
HTML5 / CSS3 / JavaScript	Frontend user interface
Jinja2	Template engine for HTML rendering
Google Gemini 2.5 Flash	Generative AI model
google-genai SDK 1.30+	Google Gemini API client
Pydantic 2.7+	Data validation and settings
python-dotenv 1.0+	Environment variable management
Git / GitHub	Version control

7. System Architecture

EduGenie follows a three-tier architecture pattern:

Layer	Component	Technology
Presentation	Web Interface	HTML5, CSS3, Vanilla JavaScript

Application	FastAPI Backend	Python FastAPI Framework
Service	Learning Modules	Python Service Classes
Integration	AI Model	Google Gemini 2.5 Flash API

Data Flow: User requests originate from the browser, traverse the FastAPI backend, are processed by appropriate service modules, and utilize the Gemini API for AI-powered responses. Responses flow back through the backend to the frontend for display.

8. System Workflow

- Step 1:** User opens EduGenie web interface in a browser
- Step 2:** User selects a learning function (Q&A, Explain, Quiz, Summarize, or Learn)
- Step 3:** User provides input (question, topic, text, or learning preferences)
- Step 4:** Frontend JavaScript validates input and sends HTTP request to FastAPI backend
- Step 5:** Backend receives request, routes to appropriate service module
- Step 6:** Service module constructs an optimized prompt for Gemini API
- Step 7:** Prompt is sent to Google Gemini 2.5 Flash with configured parameters
- Step 8:** Gemini API processes prompt and returns generated response
- Step 9:** Service module processes and validates the response
- Step 10:** Backend returns JSON response to frontend
- Step 11:** Frontend displays result in real-time to user

9. Project Structure

The EduGenie project is organized as follows:

```
EduGenie/
├── backend/
│   ├── __init__.py
│   ├── main.py (FastAPI application)
│   ├── gemini_client.py (Gemini API wrapper)
│   └── services/
│       ├── __init__.py
│       ├── qna.py (Question & Answer module)
│       ├── explanation_module.py (Topic explanation)
│       ├── quiz_module.py (Quiz generation)
│       ├── summary_module.py (Text summarization)
│       └── learning_path.py (Learning recommendations)
├── frontend/
│   ├── templates/
│   │   └── index.html (Main UI)
│   ├── static/
│   │   ├── app.js (JavaScript logic)
│   │   └── style.css (Styling)
│   └── tests/
│       └── test_api.py (API tests)
```

- .env.example (Configuration template)
- requirements.txt (Python dependencies)
- pytest.ini (Test configuration)
- README.md (Project overview)

Component	Purpose
backend/main.py	FastAPI application with 5 HTTP endpoints
backend/gemini_client.py	Handles Gemini API communication
backend/services/	Business logic for each learning function
frontend/templates/index.html	Single-page application UI
frontend/static/app.js	Frontend request handling and UI updates
frontend/static/style.css	Responsive design and styling
.env	Configuration (API keys, model selection)
requirements.txt	Python package dependencies
tests/test_api.py	Unit tests for API endpoints

10. Installation and Setup

Prerequisites

- Python 3.10 or higher installed on your system
- Git installed for version control
- A Google Gemini API key (obtain from Google AI Studio)
- A text editor or IDE (VS Code recommended)
- Internet connection for API communication

Step 1: Clone the Repository

Clone the EduGenie project from GitHub:

```
git clone https://github.com/<username>/EduGenie-Ai01.git
cd EduGenie-Ai01
```

Step 2: Create Virtual Environment

Create and activate a Python virtual environment:

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS/Linux
source .venv/bin/activate
```

Step 3: Install Dependencies

Upgrade pip and install required packages:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Step 4: Configure Environment

Copy the example environment file and add your Gemini API key:

```
copy .env.example .env # Windows
cp .env.example .env # macOS/Linux
```

Edit the .env file and replace the placeholder with your API key:

```
GEMINI_API_KEY=your_actual_api_key_here
```

Step 5: Run the Application

Start the FastAPI development server:

```
python -m uvicorn backend.main:app --reload
```

The server will start at <http://127.0.0.1:8000>

Step 6: Access the Application

Open your web browser and navigate to:

```
http://127.0.0.1:8000
```

For API documentation and testing, visit:

```
http://127.0.0.1:8000/docs
```

11. Configuration

EduGenie uses environment variables for configuration. Create a .env file in the project root with the following variables:

Variable	Purpose	Example
GEMINI_API_KEY	Google Gemini API authentication key	Alza...
GOOGLE_API_KEY	Alternative Gemini API key (fallback)	Alza...
GEMINI_MODEL	Gemini model identifier (optional)	gemini-2.5-flash

Security Recommendations

- Never commit the .env file to version control
- Store API keys in environment variables, not in source code
- Use .gitignore to exclude .env files
- Rotate API keys periodically

- Restrict API key permissions in Google Cloud console
- Use different keys for development and production

12. Features and Functionality

1. Question & Answer (Q&A;)

Purpose: Provide accurate, educational answers to student questions.

User Interaction: User selects 'Ask a Question' from the dropdown, enters their question, and submits.

Processing: The question is sent to Gemini with a prompt that emphasizes clear, concise, educational responses with examples when appropriate.

AI Usage: Gemini generates a contextual answer using temperature=0.2 (lower temperature for factuality).

Output: The answer is displayed immediately in the result panel.

2. Topic Explanation

Purpose: Simplify complex topics into beginner-friendly explanations.

User Interaction: User selects 'Explain a Topic', enters a concept or topic name, and submits.

Processing: The system constructs a prompt requesting: simple definition, how it works, an easy example, and key points.

AI Usage: Gemini generates structured explanations with temperature=0.25.

Output: A formatted explanation suitable for learners is displayed.

3. Quiz Generation

Purpose: Generate multiple-choice questions from educational materials for self-assessment.

User Interaction: User selects 'Generate Quiz', pastes educational text, specifies the number of questions (1–10), and submits.

Processing: The system sends the text to Gemini requesting JSON-formatted MCQ questions with four options and explanations.

AI Usage: Gemini generates questions with temperature=0.4 to balance creativity and relevance.

Output: A formatted quiz with questions, options, correct answers, and explanations is displayed.

4. Text Summarization

Purpose: Condense lengthy educational passages into concise summaries for quick revision.

User Interaction: User selects 'Summarize Text', pastes a passage, and submits.

Processing: The text is sent to Gemini with instructions to retain key facts and use simple language.

AI Usage: Gemini creates summaries with temperature=0.2 (prioritizing accuracy).

Output: A formatted summary is displayed, often with headings and bullet points.

5. Learning Path Recommendations

Purpose: Generate personalized, structured learning paths for new topics.

User Interaction: User selects 'Recommend Learning Path', enters a topic, selects their level (beginner, intermediate, advanced), and specifies available weekly study hours.

Processing: The system creates a personalized prompt requesting: learning stages, concepts, timeline, practice ideas, resource types, weekly routine, and milestones.

AI Usage: Gemini generates recommendations with temperature=0.35 to balance structure and creativity.

Output: A detailed, personalized learning plan is displayed in the result panel.

13. User Guide

Getting Started with EduGenie

1. Open EduGenie in your web browser (<http://127.0.0.1:8000>)
2. You will see the main interface with a task selector dropdown
3. Select one of the five learning functions: Q&A, Explain, Quiz, Summarize, or Learn
4. The input area label and help text will update based on your selection
5. Enter your question, topic, text, or learning preferences in the textarea
6. Click the 'Generate' button to submit your request
7. Wait a few seconds for the AI to process your request (status shows 'Processing')

Tips for Best Results:

- Be specific when asking questions; vague questions may receive general answers
- For explanations, provide the exact concept name or topic you want explained
- When generating quizzes, paste complete, relevant text; longer text yields better questions
- For summarization, ensure the text is coherent and educational
- When requesting learning paths, specify your current level for personalized recommendations

14. AI / Gemini Integration

Model Selection

EduGenie uses Google Gemini 2.5 Flash as the primary AI model. This model is selected for its balance of speed, cost-effectiveness, and educational quality.

API Client Implementation

The backend/gemini_client.py module manages all Gemini API communication. Key features include:

- Retrieves API key from environment variables (GEMINI_API_KEY or GOOGLE_API_KEY)
- Initializes a cached Gemini client to reduce initialization overhead
- Provides an async function for text generation with configurable parameters
- Handles API responses and error cases

Prompt Engineering

Each learning module constructs specialized prompts optimized for the target function:

Module	Temperature	Focus
Question Answering	0.2	Accuracy, factuality
Explanations	0.25	Clarity, simplicity
Quiz Generation	0.4	Variety, plausibility
Summarization	0.2	Fidelity, conciseness
Learning Paths	0.35	Structure, personalization

Temperature Parameter: Temperature controls the randomness of AI responses. Lower values (0.2) produce more focused, deterministic results. Higher values allow more creativity. EduGenie uses lower temperatures for factual tasks and moderate temperatures for creative tasks.

Response Processing

Responses from Gemini are processed by each service module before returning to the frontend. Quiz responses are parsed as JSON and validated to ensure question quality. Text responses are trimmed and formatted for display.

15. API Documentation

EduGenie exposes a RESTful API with the following endpoints:

Method	Endpoint	Purpose	Input	Output
GET	/	Serve UI	—	HTML
GET	/health	Health check	—	{status, service}
POST	/qa	Answer question	{question}	{success, result}
POST	/explain	Explain topic	{text}	{success, result}
POST	/quiz	Generate quiz	{text, count}	{success, result}
POST	/summarize	Summarize text	{text}	{success, result}
POST	/learn/recommendations	Learning path	{topic, level, weekly_hours}	{success, result}

Request/Response Schemas

Question & Answer Endpoint: POST /qa

Request:

```
{ "question": "What is photosynthesis?" }
```

Response:

```
{ "success": true, "result": "Photosynthesis is a process..." }
```

Quiz Generation Endpoint: POST /quiz

Request:

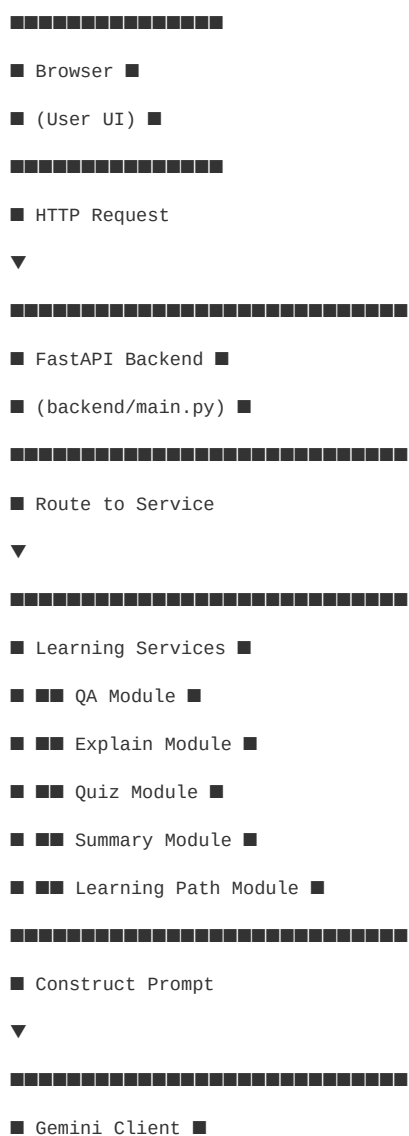
```
{ "text": "Biology passage...", "count": 3 }
```

Response:

```
{ "success": true, "result": [{"question": "...", "options": [...], "answer": "...",
"explanation": "..."}] }
```

All POST endpoints return HTTP 200 on success with status "success": true. On error, endpoints return HTTP 502 with an error message.

16. System Workflow Diagram



[illegible]

■ (gemini-2.5-flash) ■

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52

▼

[illegible]

■ & Validation ■

[illegible]

▼

[illegible]

■ (User Result) ■

[illegible]

18. Security and Privacy Considerations

Implemented Security Measures

- API keys stored in environment variables, not in source code
- Server-side API key storage prevents exposure to browser
- HTTPS recommended for production deployment
- Input validation using Pydantic models
- Appropriate HTTP status codes and error messages

Privacy Practices

EduGenie does not store user data permanently. Each session is independent, and user inputs are only transmitted to Google Gemini API for processing. No user account system or persistent database exists in the current implementation.

Recommended Production Security Practices

- Use HTTPS for all communications
- Implement rate limiting to prevent API abuse
- Add authentication if deploying to a shared environment
- Monitor API usage and costs
- Use API key rotation policies
- Enable Google Cloud logging and monitoring
- Consider containerization (Docker) for deployment
- Regular dependency updates for security patches

19. Limitations

- **API Dependency:** EduGenie requires internet connectivity and depends on Google Gemini API availability.
- **No Persistent Storage:** User sessions do not persist; history is lost upon page refresh.
- **No User Accounts:** Current version does not support user registration or personalized profiles.
- **AI Response Variability:** Generated content quality depends on Gemini's current performance; responses may occasionally contain inaccuracies.
- **Input Limits:** Maximum input lengths are configured (10,000 chars for questions, 20,000 for text).
- **Quiz JSON Dependency:** Quiz generation requires Gemini to return valid JSON; malformed responses may fail.
- **Language Support:** Currently optimized for English; other languages may not be fully supported.
- **No Offline Mode:** Application requires live API connectivity; offline use is not possible.
- **Browser Compatibility:** Client-side JavaScript requires modern browser versions.
- **Cost Considerations:** Google Gemini API usage incurs costs based on token consumption.

20. Future Enhancements

The following features are recommended for future development phases:

- **User Accounts & Authentication:** Implement user registration, login, and personalized user profiles.
- **Learning History & Progress Tracking:** Store user interactions and learning progress in a database.
- **Persistent Database:** Add a database (PostgreSQL/MongoDB) for storing learning histories and user data.
- **Mobile Application:** Develop native iOS and Android applications for mobile learning.
- **Multi-Language Support:** Implement language detection and support for multiple languages.
- **Advanced Analytics:** Track learning patterns, quiz performance, and personalized recommendations.
- **Local Model Support:** Integrate open-source models (Mistral, Llama) for offline capabilities.
- **Video Content Generation:** Extend capabilities to generate educational video scripts.
- **Adaptive Learning Paths:** Create dynamic learning paths based on quiz performance.
- **Social Features:** Add collaborative learning, study groups, and knowledge sharing.
- **Integration with LMS:** Connect with platforms like Moodle, Canvas, or Google Classroom.
- **Content Caching:** Implement response caching to reduce API costs and improve response times.
- **Advanced Prompt Templates:** Create subject-specific prompt templates for specialized domains.
- **Accessibility Improvements:** Enhance WCAG compliance for users with disabilities.

21. Conclusion

EduGenie successfully demonstrates the application of Google Gemini's generative AI capabilities to educational technology. The project provides a clean, functional, and user-friendly learning assistant that leverages modern web technologies and AI APIs.

The system architecture follows best practices with clear separation between frontend and backend components, secure API key management, and modular service design. The five learning functions—question answering, topic explanations, quiz generation, text summarization, and learning path recommendations—collectively provide comprehensive support for diverse learning needs.

Through FastAPI's performance and Gemini's language understanding capabilities, EduGenie delivers responsive, contextual, and educational assistance. The project successfully meets the objectives of Phase 7 (Project Documentation) by providing complete technical documentation suitable for SmartBridge submission, GitHub publication, and future development.

Future enhancements—including user accounts, persistent storage, mobile applications, and advanced analytics—can extend EduGenie into a comprehensive educational platform. The current implementation provides a solid foundation for these expansions while demonstrating core competencies in full-stack development, AI integration, and educational technology.

Project Status: EduGenie is fully functional and ready for demonstration (Phase 8). The application successfully integrates Google Gemini, implements all specified learning functions, and provides a polished user experience. This documentation completes Phase 7 of the SmartBridge development lifecycle.



